

Virtual Worker Platform (VWP)

Prompt Library & UI Code Templates

Version: 1.0

Scope: MVP (Finance, Technical Ops, Sales & Marketing, Logistics, Developer) Architecture Alignment: AI → Governance → n8n Execution → Audit

PART 1: PROMPT LIBRARY (HUGGING FACE / AI GATEWAY)

This prompt library is designed to support the template-constrained execution model defined in the MVP Definition and Technical Design documents. The AI layer MUST:

- Output strict JSON only
 - Follow template input schemas
 - Never execute actions directly
 - Never fabricate capabilities
 - Respect approval and risk constraints
-

1. Global System Prompt (All Domains)

You are the AI Interpretation Layer of the Virtual Worker Platform.

Your role:

1. Classify the user's request into one approved template.
2. Extract structured inputs strictly matching the provided JSON schema.
3. Do NOT execute actions.
4. Do NOT invent fields.
5. If information is missing, return "needs_clarification".
6. Output must be valid JSON only.

If confidence < threshold, set "confidence": "low".

Never return prose outside JSON.

2. Classification Prompt Template

```
User Task:
"{user_input}"

Available Templates:
{template_list}

Instructions:
- Choose ONE best matching template.
- Provide confidence score (0-1).
- If no suitable template, return "no_match".

Output JSON:
{
  "selected_template": "template_name",
  "confidence": 0.00,
  "rationale": "short explanation"
}
```

DOMAIN PROMPTS

FINANCE DOMAIN

Expense Summary Generator

Extraction Prompt:

```
Extract the following fields:
- date_range_start (ISO date)
- date_range_end (ISO date)
- receipt_ids (array)
- output_format (pdf|excel)

Return strictly valid JSON.
```

Expected Output Shape:

```
{
  "date_range_start": "2026-01-01",
```

```
"date_range_end": "2026-01-31",
"receipt_ids": ["att_123"],
"output_format": "pdf"
}
```

TECHNICAL OPERATIONS

Incident Summary Generator

Prompt:

```
From the provided logs, summarize:
- system_name
- time_window
- severity_level
- affected_services
```

```
Return structured JSON only.
```

SALES & MARKETING

Draft Outreach Email (Approval Required)

Prompt:

```
Generate a professional outreach email draft.
```

```
Inputs required:
```

- ```
- recipient_name
- company
- value_proposition
- call_to_action
```

```
Do NOT send email. Only draft.
```

```
Output JSON:
```

```
{
 "subject": "...",
 "body": "..."
}
```

## LOGISTICS

### Shipment Status Query

Prompt:

```
Extract:
- tracking_number
- carrier
```

```
Return JSON only.
```

---

## DEVELOPER DOMAIN

### Pull Request Summary

Prompt:

```
Analyze the provided PR diff.
Return JSON:
{
 "summary": "...",
 "files_changed": number,
 "risk_level": "low|medium|high",
 "requires_review": true|false
}
```

---

## PART 2: UI CODE TEMPLATES (React + TypeScript)

Architecture: API-first (vwp-api), JWT auth, workspace scoped

---

### 1. Task Submission Component

```
import { useState } from "react";

export default function TaskSubmit() {
 const [description, setDescription] = useState("");
 const [loading, setloading] = useState(false);
```

```

const submitTask = async () => {
 setLoading(true);
 await fetch("/api/v1/tasks", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ description })
 });
 setLoading(false);
};

return (
 <div className="p-4">
 <textarea
 className="border w-full p-2"
 value={description}
 onChange={(e) => setDescription(e.target.value)}
 placeholder="Describe your task..."
 />
 <button
 onClick={submitTask}
 className="bg-blue-600 text-white px-4 py-2 mt-2"
 >
 {loading ? "Submitting..." : "Submit Task"}
 </button>
 </div>
);
}

```

## 2. Approval Queue Component

```

export function ApprovalQueue({ approvals }) {
 return (
 <div>
 {approvals.map((a) => (
 <div key={a.id} className="border p-3 mb-2">
 <h3 className="font-bold">{a.scope}</h3>
 <p>Status: {a.status}</p>
 <button className="bg-green-500 text-white px-2 py-1 mr-2">Approve</button>
 <button className="bg-red-500 text-white px-2 py-1">Reject</button>
 </div>
))}
 </div>
);
}

```

### 3. Task Status Dashboard

```
export function TaskDashboard({ tasks }) {
 return (
 <table className="w-full border">
 <thead>
 <tr>
 <th>ID</th>
 <th>Status</th>
 <th>Template</th>
 <th>Created</th>
 </tr>
 </thead>
 <tbody>
 {tasks.map((t) => (
 <tr key={t.id}>
 <td>{t.id}</td>
 <td>{t.status}</td>
 <td>{t.template}</td>
 <td>{t.created_at}</td>
 </tr>
))}
 </tbody>
 </table>
);
}
```

## PART 3: TEMPLATE CONFIG JSON EXAMPLE

```
{
 "template_id": "finance_expense_summary",
 "risk_level": "medium",
 "allowed_actions": ["generate_report"],
 "approval_required": true,
 "input_schema": {
 "type": "object",
 "properties": {
 "date_range_start": { "type": "string" },

```

```
 "date_range_end": { "type": "string" }
 },
 "required": ["date_range_start", "date_range_end"]
}
```

---

**END OF DOCUMENT**